Integración de datos

Tarea Práctica - Clase 7

Sergio Enrique Morel Peralta
Facultad Politécnica - UNA, 12.04.2026

1. Tarea 7: Orquestación y Visualización

Esta tarea implementa el pipeline ELT completo para el dataset **Maven Fuzzy Factory**, un ecommerce ficticio con datos de sesiones web, órdenes, productos y reembolsos. El pipeline sigue la arquitectura: **MySQL** -> **Airbyte** -> **MotherDuck** -> **dbt** -> **Metabase**, orquestado con **Prefect**. El workspace se ubica en `workspaces/maven-fuzzy/` dentro del mismo repositorio de las tareas anteriores.

1.1. Paso 1: Base de datos MySQL con Docker

1.1.1. Dataset: Maven Fuzzy Factory

El dataset proviene de **Maven Analytics Data Playground** y contiene la base de datos transaccional de un ecommerce de juguetes ficticio. Se descarga como archivo `.zip` y se carga en una instancia local de MySQL.

Tabla	Registros	Descripción
`website_sessions`	472,871	Sesiones de usuarios con datos UTM, device y referer
`website_pageviews`	1,188,124	Páginas visitadas por sesión
`orders`	32,313	Órdenes realizadas con precio y costo

Tabla	Registros	Descripción
`order_items`	40,025	Items individuales dentro de cada orden
`order_item_refunds`	1,731	Reembolsos procesados
`products`	4	Catálogo de productos

1.1.2. Contenedor Docker

Se levanta MySQL 8.0 junto a phpMyAdmin mediante `docker-compose.yaml` en `workspaces/maven-fuzzy/containers/`. Las credenciales se leen desde un archivo `.env` (no versionado); el archivo `example.env` sirve de plantilla:

```
# Crear .env a partir de la plantilla
cd workspaces/maven-fuzzy/containers
cp example.env .env
# Editar .env con las credenciales reales
```

```yaml
services:
  mia-mysql:
    image: mysql:8.0
    container_name: mia-mysql
    restart: unless-stopped
    environment:
      - MYSQL_ROOT_PASSWORD
      - MYSQL_DATABASE
      - MYSQL_USER
      - MYSQL_PASSWORD
    ports:
      - "${MYSQL_PORT}:3306"
    expose:
      - "3306"
    volumes:
      - mia-mysql_data:/var/lib/mysql:rw
      - ./initdb:/docker-entrypoint-initdb.d:ro
      - ${CSV_DIR}:/csv
      - /etc/localtime:/etc/localtime:ro
    command: >
      --default-authentication-plugin=mysql_native_password
      --local-infile=1
      --secure-file-priv=/csv

  mia-phpmyadmin:
    image: phpmyadmin:latest
    container_name: mia-phpmyadmin
    restart: unless-stopped
    environment:
      - PMA_HOST=mia-mysql
      - PMA_USER=${MYSQL_USER}
      - PMA_PASSWORD=${MYSQL_PASSWORD}
```

```yaml
    ports:
      - "${PMA_PORT}:80"
    expose:
      - "80"
    volumes:
      - /etc/localtime:/etc/localtime:ro
    depends_on:
      - mia-mysql

volumes:
  mia-mysql_data:
    name: mia-mysql_data
```

La opción `--default-authentication-plugin=mysql_native_password` en MySQL garantiza compatibilidad con el conector de Airbyte, que no soporta el plugin `caching_sha2_password` introducido por defecto en MySQL 8.0.

phpMyAdmin se pre-configura automáticamente con `PMA_HOST`, `PMA_USER` y `PMA_PASSWORD`, por lo que no requiere ingresar credenciales manualmente al acceder a `localhost:8095`. El puerto 8095 se eligió para evitar conflicto con `dbt docs serve`, que usa 8080 por defecto.

1.1.3. Schema de la base de datos

El schema se declara en `initdb/01_schema.sql` siguiendo los tipos de datos del modelo original. MySQL ejecuta este script automáticamente al primer arranque del contenedor.

Tabla	Columnas clave y tipos
products	product_id INT, product_name VARCHAR(45)
website_sessions	website_session_id BIGINT, is_repeat_session BINARY(1), utm_source/campaign/content VARCHAR(45)
website_pageviews	website_pageview_id BIGINT, pageview_url VARCHAR(45)
orders	order_id BIGINT, price_usd DECIMAL(6,2), cogs_usd DECIMAL(6,2)
order_items	order_item_id BIGINT, is_primary_item BINARY(1), price_usd DECIMAL(6,2)
order_item_refunds	order_item_refund_id BIGINT, refund_amount_usd DECIMAL(6,2)

1.1.4. Carga de datos

Los CSVs se montan en `/csv` dentro del contenedor a través del volumen `${CSV_DIR}:/csv`. El script `initdb/02_load_data.sql` usa `LOAD DATA INFILE` para cargar cada archivo:

```sql
LOAD DATA INFILE '/csv/website_sessions.csv'
INTO TABLE website_sessions
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(website_session_id, created_at, user_id, is_repeat_session,
 utm_source, utm_campaign, utm_content, device_type, http_referer);
```

La variable `CSV_DIR` en `.env` apunta a la carpeta local donde se descomprimió el `.zip` descargado de Maven Analytics Data Playground.

1.1.5. Levantar los contenedores

```bash
cd workspaces/maven-fuzzy/containers
cp example.env .env
# Editar .env: ajustar CSV_DIR y credenciales
docker compose up -d
```

Al primer arranque MySQL ejecuta los scripts de `initdb/` en orden numérico: primero crea el schema (`01_schema.sql`) y luego carga los datos (`02_load_data.sql`). La carga de las tablas más grandes (`website_sessions` con 450K filas, `website_pageviews` con 1.1M) puede tomar algunos minutos. El progreso es visible en los logs del contenedor:

```bash
docker logs -f mia_mysql
```

1.1.6. Verificación

Una vez que el contenedor está corriendo, se puede verificar la carga desde phpMyAdmin (`localhost:8095`) ejecutando:

```sql
SELECT 'products'            AS tabla, COUNT(*) AS registros FROM
products
UNION ALL SELECT 'orders',                        COUNT(*) FROM orders
UNION ALL SELECT 'order_items',                   COUNT(*) FROM
order_items
UNION ALL SELECT 'order_item_refunds',            COUNT(*) FROM
order_item_refunds
UNION ALL SELECT 'website_sessions',              COUNT(*) FROM
website_sessions
UNION ALL SELECT 'website_pageviews',             COUNT(*) FROM
website_pageviews;
```

Resultado obtenido:

Tabla	Registros
`products`	4
`orders`	32,313

Tabla	Registros
`order_items`	40,025
`order_item_refunds`	1,731
`website_sessions`	472,871
`website_pageviews`	1,188,124

1.2. Paso 2: Configurar Airbyte

Se configuraron tres componentes en Airbyte (`localhost:8000`):

1.2.1. Source: MySQL

Campo	Valor
Nombre	`MySQL_Maven-Fuzzy-Factory`
Host	IP ZeroTier de la notebook Linux
Port	`3306`
Database	`maven_fuzzy_factory`
Username	`airbyte`

1.2.2. Destination: MotherDuck

Campo	Valor
Nombre	`MotherDuck_maven_fuzzy`
Database	`md:airbyte_curso`
Default schema	`maven_fuzzy`

1.2.3. Connection

Campo	Valor
Source	`MySQL_Maven-Fuzzy-Factory`
Destination	`MotherDuck_maven_fuzzy`
Sync mode	Full refresh | Overwrite
Schedule	Manual (orquestado por Prefect)
Tablas sincronizadas	`website_sessions`, `website_pageviews`, `orders`, `order_items`, `order_item_refunds`, `products`
Connection ID	`39cdf568-8a26-4c2e-95fc-b6bc0dc989a4`

Las tablas se sincronizan al schema `airbyte_curso.maven_fuzzy` en MotherDuck. El schedule se configura como Manual dado que la ejecución será orquestada por Prefect. El primer sync completó exitosamente, dejando las 6 tablas disponibles en MotherDuck.

1.3. Paso 3: Modelos dbt

El proyecto dbt se inicializó en `workspaces/maven-fuzzy/dbt_maven_fuzzy/` con `dbt init`. Los modelos transforman los datos crudos del schema `maven_fuzzy` (cargados por Airbyte) en dos capas:

Schema destino	Materialización	Descripción
`maven_fuzzy_staging`	view	Limpieza y estandarización de tablas fuente
`maven_fuzzy_marts`	table	Modelos analíticos agregados para visualización

1.3.1. Configuración

El archivo `profiles.yml` define la conexión a MotherDuck y el schema base `maven_fuzzy`, que dbt usa como prefijo para generar los nombres de schema destino:

```yaml
dbt_maven_fuzzy:
  outputs:
    dev:
      type: duckdb
      path: "md:airbyte_curso"
      schema: maven_fuzzy
      motherduck_token: "{{ env_var('MOTHERDUCK_TOKEN') }}"
  target: dev
```

El `dbt_project.yml` configura las materializaciones y sufijos de schema:

```yaml
models:
  dbt_maven_fuzzy:
    staging:
      +materialized: view
      +schema: staging
    marts:
      +materialized: table
      +schema: marts
```

Las variables de entorno se configuran con `source set_env.sh` antes de ejecutar dbt (el archivo `set_env.example.sh` sirve de plantilla).

1.3.2. Modelos staging

Los modelos staging leen desde `{{ source('raw', ...) }}` apuntando al schema `airbyte_curso.maven_fuzzy`. Cada modelo limpia y tipifica los campos de su tabla fuente.

Nota sobre campos BINARY(1): Airbyte serializa las columnas `BINARY(1)` de MySQL (como `is_primary_item` e `is_repeat_session`) como strings base64 en MotherDuck. Los valores posibles son `'MA=='` (0) y `'MQ=='` (1). Los modelos staging convierten estos valores a booleanos comparando directamente el string base64:

```
-- stg_order_items.sql (fragmento)
oi.is_primary_item = 'MQ==' as is_primary_item
```

Modelo	Transformaciones principales
`stg_sessions`	Reemplaza `utm_source` null por `'direct'`, convierte `is_repeat_session` de base64 a boolean, agrega truncados de fecha (día, semana, mes, año)
`stg_orders`	Calcula `margin_usd = price_usd - cogs_usd`, agrega truncados de fecha
`stg_order_items`	Join con `products` para traer `product_name`, convierte `is_primary_item` de base64 a boolean, calcula margen por ítem
`stg_pageviews`	Selecciona campos relevantes, agrega `pageview_date`
`stg_refunds`	Selecciona campos relevantes, agrega `refund_date`

1.3.3. Modelos marts

Modelo	Descripción
`obt_orders_enriched`	One Big Table: join de órdenes con sesiones, productos, ítems agregados y reembolsos. Incluye campos calculados como `channel_group`, `order_tier` y `net_revenue`
`fct_daily_sales`	Métricas por día: sesiones, órdenes, conversión, revenue, cogs, margen y reembolsos
`fct_channel_performance`	Performance por canal UTM, campaña y device: sesiones, órdenes, conversión, revenue y AOV
`fct_product_performance`	Performance por producto: unidades vendidas, revenue, margen, tasa de reembolso

1.3.4. DAG

El grafo de dependencias del modelo `obt_orders_enriched` muestra cómo las cinco tablas fuente fluyen a través de los modelos staging antes de consolidarse en el mart:

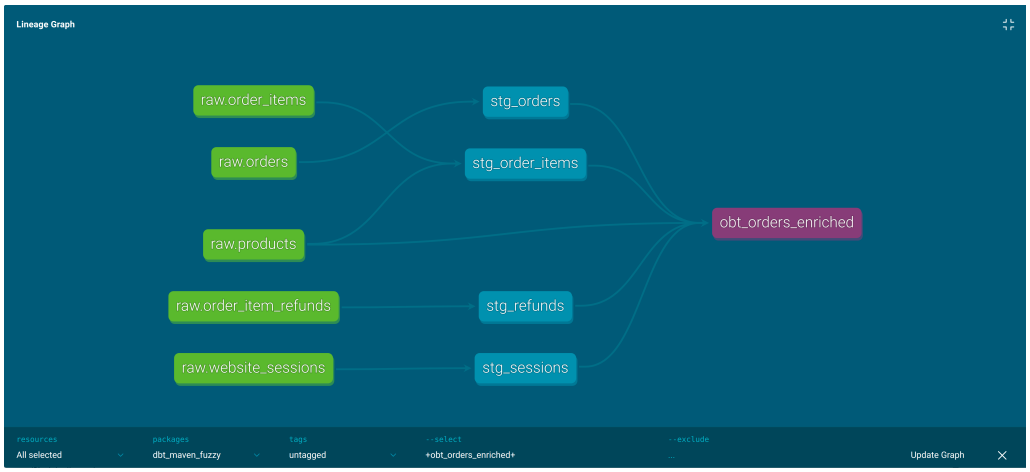

Figura 1: DAG de `obt_orders_enriched` generado con `dbt docs`

1.3.5. Ejecución

```
cd workspaces/maven-fuzzy/dbt_maven_fuzzy
source set_env.sh
dbt deps
dbt run
dbt test
```

Resultado de `dbt run`: **PASS=9, ERROR=0** (5 views en staging + 4 tables en marts).

Resultado de `dbt test`: **PASS=20, ERROR=0** (tests de unicidad, not_null, valores aceptados y singular test de revenue positivo).

1.4. Paso 4: Orquestación con Prefect

El pipeline se orquesta con Prefect 3, que coordina la ejecución secuencial de las tres etapas: sincronización con Airbyte, transformación con dbt y validación con tests.

1.4.1. Estructura del pipeline

El flow `ecommerce_pipeline` se define en `workspaces/maven-fuzzy/prefect/ecommerce_pipeline.py` y contiene tres tasks:

Task	Nombre	Descripción
1	`Extract and Load with Airbyte`	Dispara el sync de Airbyte via API REST y espera hasta que el job termine (polling cada 10s). Reintentos: 2
2	`Transform with dbt`	Ejecuta `dbt deps` + `dbt run` como subproceso
3	`Test with dbt`	Ejecuta `dbt test` como subproceso

1.4.2. Configuración

Las variables de entorno se cargan desde `prefect/.env` (no versionado; `prefect/.env.example` sirve de plantilla):

```
AIRBYTE_HOST=localhost
AIRBYTE_PORT=8000
AIRBYTE_USERNAME=airbyte
AIRBYTE_PASSWORD=password
AIRBYTE_CONNECTION_ID=39cdf568-8a26-4c2e-95fc-b6bc0dc989a4
MOTHERDUCK_TOKEN=TU_TOKEN
```

Las dependencias con versiones fijas están en `workspaces/maven-fuzzy/requirements.txt`. Se usa `prefect==3.3.7` ya que Prefect 3.6.x tiene un bug de compatibilidad con `fakeredis` que impide levantar el servidor.

1.4.3. Ejecución

```
# Terminal 1: servidor Prefect
prefect server start

# Terminal 2: pipeline
prefect config set PREFECT_API_URL=http://127.0.0.1:4200/api
cd workspaces/maven-fuzzy/prefect
python ecommerce_pipeline.py
```

1.4.4. Resultado

El pipeline completó exitosamente con las tres tasks en estado `Completed`:

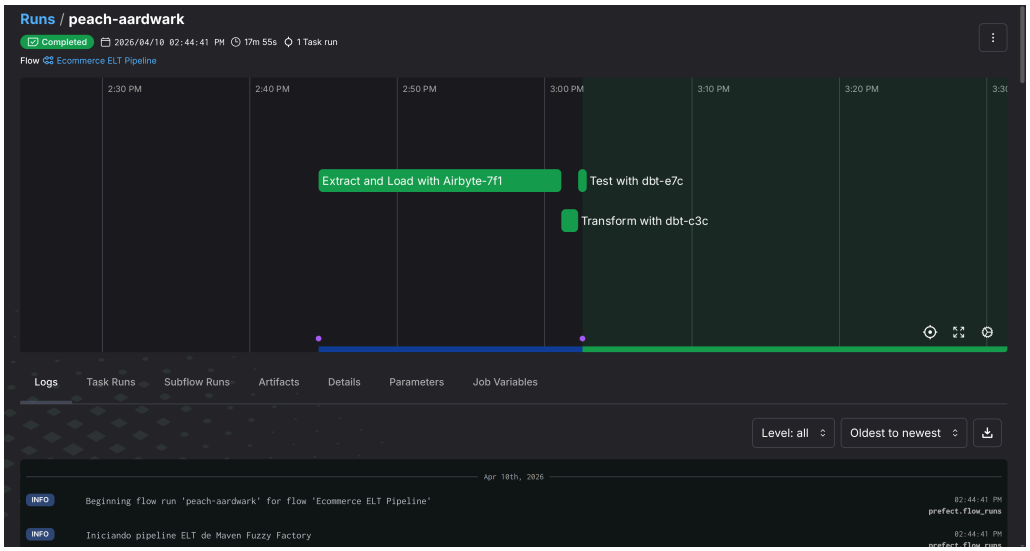

Figura 2: Prefect UI — flow run `peach-aardwark` completado

1.5. Paso 5: Visualización con Metabase

Metabase se levanta como contenedor Docker junto a MySQL y phpMyAdmin en `workspaces/maven-fuzzy/containers/`.

1.5.1. Imagen Docker

Se construye una imagen personalizada que combina Metabase con el driver DuckDB de la comunidad, definida en `containers/Dockerfile`:

```
FROM eclipse-temurin:21-jre
ENV MB_PLUGINS_DIR=/plugins
RUN mkdir -p ${MB_PLUGINS_DIR} /app
# Metabase v0.58.8
ADD https://downloads.metabase.com/v0.58.8/metabase.jar /app/
metabase.jar
# DuckDB driver para Metabase
ADD https://github.com/motherduckdb/metabase_duckdb_driver/releases/
download/1.4.3.1/duckdb.metabase-driver.jar ${MB_PLUGINS_DIR}/
EXPOSE 3000
CMD ["java", "-jar", "/app/metabase.jar"]
```

El servicio `mia-metabase` se agrega al `docker-compose.yaml` existente, referenciando el `Dockerfile` directamente con `build: context: .`. El volumen `mia-metabase_data` persiste la configuración y los dashboards entre reinicios.

1.5.2. Levantar el servicio

```
cd workspaces/maven-fuzzy/containers
cp example.env .env
# Editar .env con las credenciales reales

# Primera vez: construir la imagen
docker compose build mia-metabase

# Levantar
docker compose up -d mia-metabase
```

1.5.3. Conexión a MotherDuck

En el setup inicial de Metabase (`localhost:3000/setup`), al agregar la base de datos se selecciona el driver **DuckDB** y se configura:

Campo	Valor
Nombre para mostrar	`Maven-Fuzzy`
Database file	`md:airbyte_curso?motherduck_token=TU_TOKEN`
Use DuckDB old_implicit_casting option	activado

El token se incluye directamente en el campo **Database file** ya que el driver DuckDB no tiene campo separado para autenticación.

1.5.4. Dashboard

El dashboard **Maven Fuzzy Factory** contiene cinco visualizaciones construidas con Native Query (SQL) sobre los modelos marts:

#	Tipo	Descripción
1	Number	KPIs principales: Total Orders, Revenue, Margin, AOV — desde `obt_orders_enriched`
2	Line chart	Revenue y margen por mes — desde `obt_orders_enriched`
3	Bar chart	Revenue por producto — desde `obt_orders_enriched`
4	Table	Conversión por canal UTM y device — desde `fct_channel_performance`
5	Line chart	Ventas diarias: órdenes, revenue y tasa de conversión — desde `fct_daily_sales`

Los filtros se implementan con variables `{{campo}}` en las queries SQL (Field Filter de Metabase):

Filtro	Tipo	Cards vinculadas
Fecha	Date Range	KPIs, Revenue por Mes, Ventas Diarias, Performance por Canal
Utm Source	Text	KPIs, Revenue por Mes, Ventas Diarias, Performance por Canal

Figura 3: Dashboard Maven Fuzzy Factory en Metabase — vista completa sin filtros

Figura 4: Dashboard filtrado por fecha (12/3/2013 – 11/4/2014) y canal (`direct`)